Ingeniería de Software · Evaluación 3

# Sistema de ventas e inventario para El Matute

Propuesta tecnológica, análisis ético y legal, UML, prototipo seguro y pruebas

Caso de estudio: El Matute, emprendimiento de ropa interior. El prototipo aborda el control de existencias por producto y talla, el registro de ventas y la actualización del stock.

| | |
|---|---|
| Equipo | Dos integrantes (roles identificados en sección 4) |
| Fecha | 20 de octubre de 2026 |
| Evaluación | EVA3 - 40% de la asignatura |

Informe académico basado en el alcance funcional demostrado y en los antecedentes del caso El Matute.

# Resumen ejecutivo

El Matute necesita reemplazar planillas y registros manuales por un sistema pequeño y confiable que mantenga al día el stock por prenda y talla, registre ventas y permita recuperar el estado del inventario después de errores. El prototipo usa Flask y SQLite para demostrar el flujo crítico en un solo local. Para crecer, se propone desplegar la aplicación en una plataforma administrada y migrar la base a PostgreSQL administrado.

La venta descuenta existencias dentro de una transacción: si el producto no existe o no hay stock suficiente, la operación se rechaza sin crear una venta parcial. La solución evita recopilar datos personales de clientes en esta etapa; los registros de venta contienen producto, talla, cantidad, precio y fecha. El historial ofrece trazabilidad básica, pero no es un registro inmutable ni una auditoría protegida contra cambios directos en la base de datos. La versión de entrega incorpora inicio de sesión, token CSRF, consultas parametrizadas, límites de entrada y cabeceras de seguridad.

# 1. Contexto y alcance

El caso se basa en el emprendimiento de ropa interior El Matute y en la aplicación Flask disponible para la evaluación. Por el tipo de catálogo, una misma prenda puede variar por modelo y talla; confirmar esa disponibilidad al vender y conservar un registro claro de cada movimiento son necesidades centrales del caso. El prototipo demuestra esas funciones en una operación de una sucursal.

## Problema y objetivos

| Necesidad operacional | Respuesta del prototipo | Criterio de aceptación |
|---|---|---|
| Confirmar disponibilidad por modelo y talla para evitar sobreventa. | Catálogo con producto, talla, precio y stock; selector muestra disponibilidad. | El inventario refleja el nuevo stock tras una venta válida. |
| Evitar errores al calcular cobros y reconstruir movimientos. | Calcular el total y guardar cantidad, precio unitario, total y fecha. | El total coincide con precio × cantidad y la venta aparece en el historial. |
| Restringir cambios y reducir solicitudes no autorizadas. | Exigir sesión, token CSRF y validación de entradas. | Peticiones sin sesión o CSRF se rechazan. |

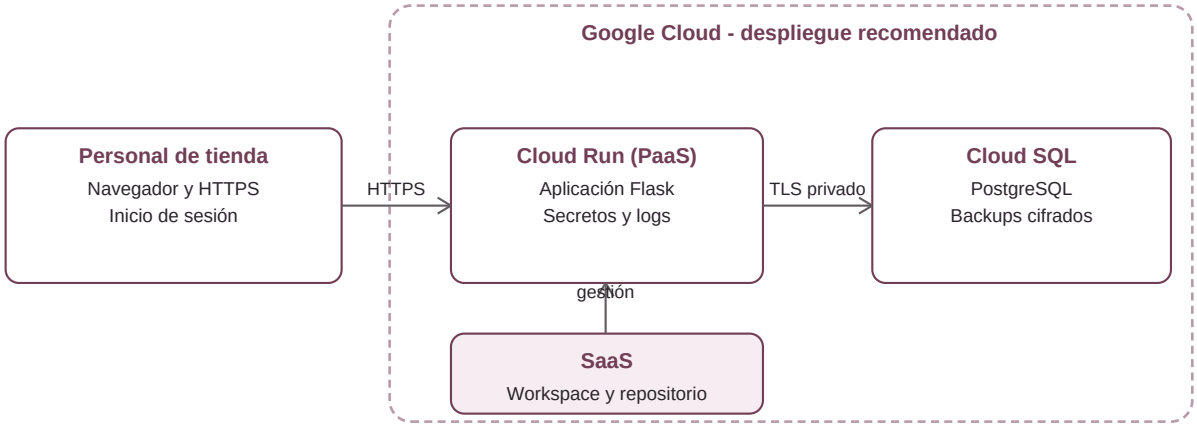
## Alcance funcional

Incluye acceso de administrador, alta de producto, consulta de inventario, registro de venta, validación de stock y vista de las últimas diez ventas. No incluye pago electrónico, facturación tributaria, devoluciones, clientes, varias sucursales ni sincronización con un POS externo. Es un prototipo académico; no debe tratarse como sistema productivo certificado.

# 2. Propuesta tecnológica e infraestructura cloud

El diseño mantiene una aplicación web simple para el equipo de tienda. El navegador se comunica por HTTPS con Flask. La aplicación administra las reglas de venta; una base relacional conserva productos y movimientos. Para el prototipo, SQLite reduce complejidad y facilita probar el descuento de stock en una sucursal. En una operación desplegada con varios usuarios, PostgreSQL administrado mejora la concurrencia, el respaldo y la operación.

## Arquitectura propuesta




*IaaS (Compute Engine) es alternativa si se necesita controlar el sistema operativo; exige más administración.*

| Modelo | Uso propuesto en El Matute | Ventaja y límite |
|---|---|---|
| Cloud Computing | Uso de recursos de cómputo, red y almacenamiento bajo demanda; la arquitectura separa aplicación, base y respaldos. | Reduce inversión inicial y permite ajustar capacidad; requiere conectividad, gestión de accesos y control de costos. |
| SaaS | Google Workspace para correo y documentos; repositorio GitHub/GitLab como servicio de colaboración. | No hay que instalar ni mantener esos productos; revisar ubicación de datos, permisos y condiciones del proveedor. |
| PaaS | Cloud Run para publicar el servicio Flask sin administrar servidores; Cloud SQL para PostgreSQL administrado. | El proveedor mantiene plataforma y parches del servicio; la PYME sigue responsable de código, permisos y datos. |
| IaaS | Compute Engine como alternativa si se necesita controlar el sistema operativo o instalar componentes propios. | Mayor flexibilidad, pero la tienda tendría que mantener sistema operativo, firewall, parches y recuperación. |

## Justificación y operación

Para un emprendimiento pequeño de ropa interior, la prioridad es controlar las existencias por modelo y talla, evitar diferencias de stock y recuperar respaldos sin personal dedicado a servidores. Por eso, la ruta recomendada es PaaS y base administrada; IaaS queda como alternativa condicionada por una necesidad técnica concreta. Cloud SQL ofrece administración del motor relacional y opciones de cifrado y respaldo, sujetas a configuración del servicio. Antes de contratar se compararán costo mensual total, región disponible, exportación y restauración de datos, soporte, SLA, límites de uso y dependencia del proveedor.

Operación y continuidad: definir con la dueña el tiempo máximo tolerable sin servicio y pérdida de datos; programar respaldos diarios y probar restauración trimestral. Si falla Internet, anotar ventas con folio y conciliarlas al restablecer conexión. Proteger con HTTPS, secretos fuera del código, red privada, MFA y alertas de presupuesto.

# 3. Confiabilidad, ética y marco legal

La confiabilidad protege la continuidad de caja y la confianza de quienes administran el inventario. Una caída durante una venta puede duplicar un cobro, dejar el stock desactualizado o retrasar la atención. Una pérdida de datos puede impedir reponer prendas o explicar diferencias. El diseño usa una actualización condicional de stock y registra la venta dentro de una transacción, evitando confirmar operaciones incompletas.

## Principios éticos aplicados

El equipo limita el acceso a personas autorizadas, minimiza los datos recogidos, explica a la PYME qué se almacena, protege la información y prueba solo el sistema propio o con autorización. El código de ética de ACM orienta a priorizar el bien público, respetar privacidad y diseñar seguridad usable. Los resultados de pruebas deben comunicarse con honestidad; una suite aprobada no equivale a una auditoría de seguridad completa.

## Marco chileno relevante a la fecha del informe

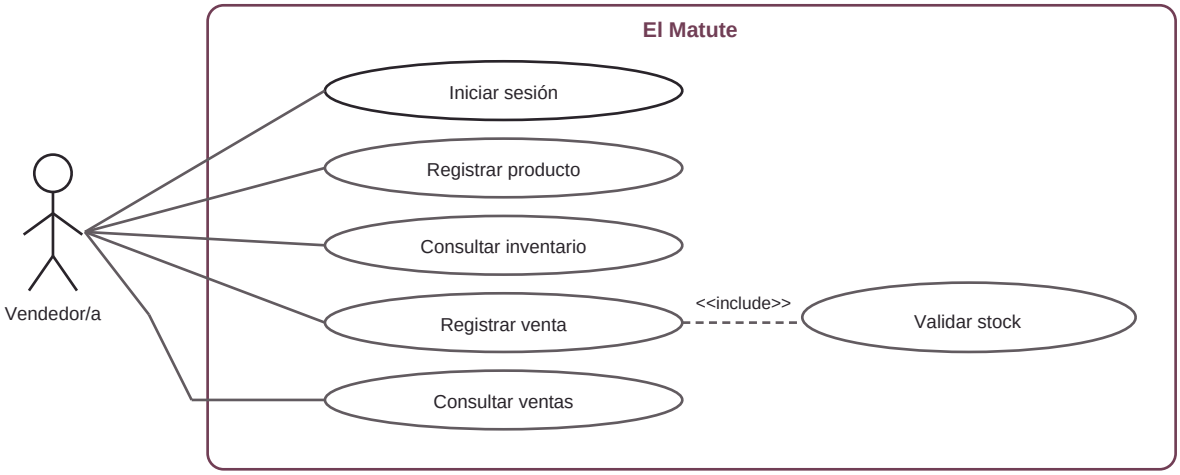
| Norma | Aplicación al caso |
|---|---|
| Ley 19.628 sobre protección de la vida privada (vigente hasta 30-11-2026). | Si se incorporan datos personales de clientes o trabajadores, definir finalidad y base aplicable, restringir accesos, proteger registros y responder solicitudes conforme al texto vigente. |
| Ley 21.719, publicada en 2024; modificaciones con vigencia desde 01-12-2026. | Reforma el régimen de datos personales y crea una autoridad de protección de datos. Diseñar desde ahora con minimización, finalidad, seguridad y atención de derechos para facilitar la transición. |
| Ley 21.459 sobre delitos informáticos (texto vigente consultado al 01-04-2025). | El acceso no autorizado, ataques a integridad, interceptación y otras conductas descritas en la ley son un riesgo jurídico. Las pruebas deben limitarse al prototipo y a entornos autorizados. |

En el prototipo no se solicitan ni almacenan nombres, RUT, teléfonos, correos ni medios de pago de clientes. La PYME debe validar el tratamiento de datos y sus obligaciones con asesoría competente antes de operar con datos reales. Este apartado identifica normas aplicables al caso, pero no sustituye una evaluación jurídica.

# 4. Modelado UML y trabajo en equipo

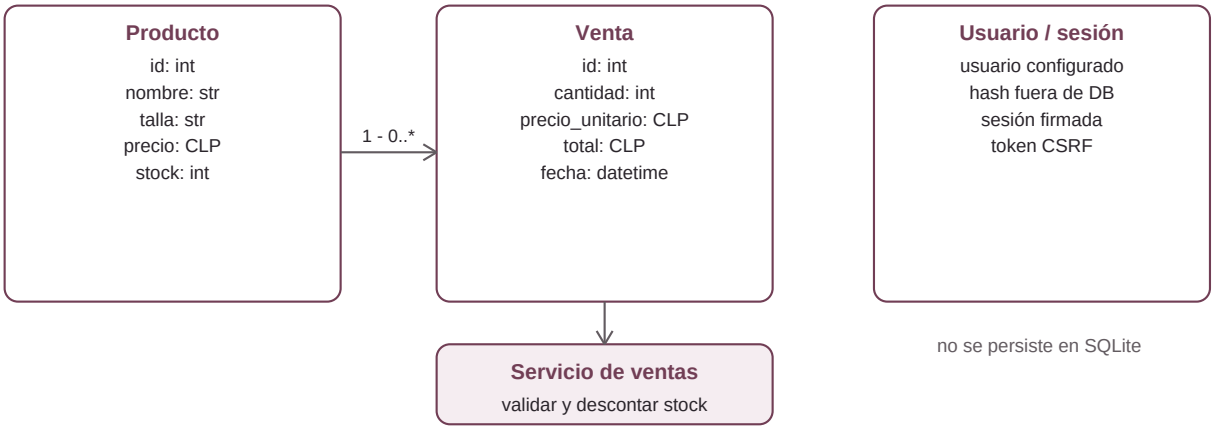
Los diagramas describen al vendedor como actor principal y el flujo crítico como una venta que valida sesión y CSRF, intenta descontar stock y guarda el movimiento solo cuando existe disponibilidad.

## Diagrama de casos de uso




El caso de uso «Registrar venta» incluye «Validar stock». El actor consulta catálogo y ventas, registra productos y termina la sesión. La autorización ocurre antes de ejecutar acciones de cambio.

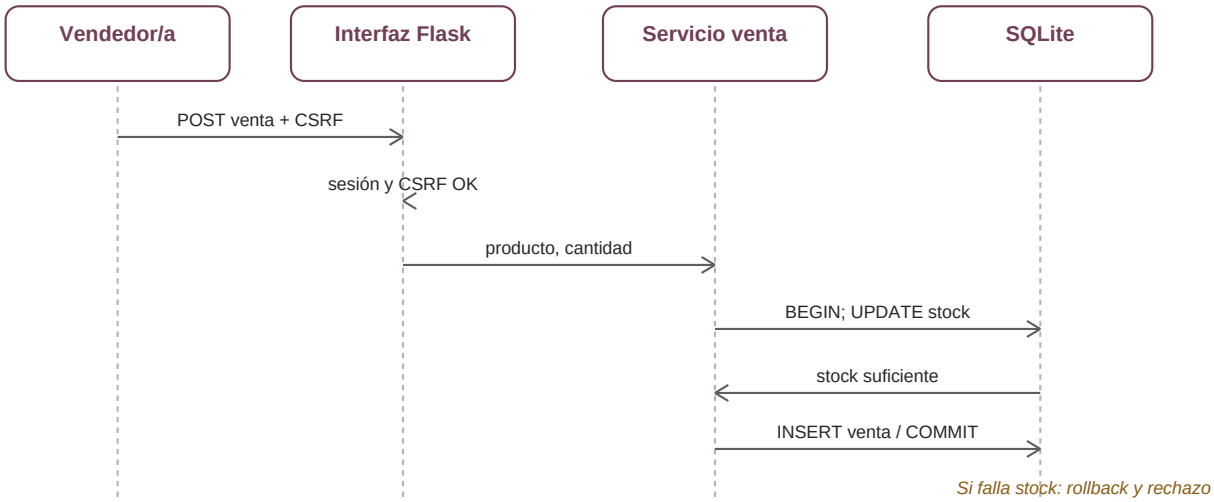
## Diagrama de clases




Una venta referencia un producto; el servicio de ventas coordina la operación transaccional. El usuario se configura fuera de SQLite y su sesión autoriza el acceso; por eso no aparece como una tabla persistente.

# 4. Modelado UML y trabajo en equipo (continuación)

## Secuencia de venta




La operación comienza con la sesión y el token CSRF válidos. SQLite inicia una transacción, actualiza el stock solo cuando alcanza para toda la cantidad y guarda el precio aplicado. Si no se cumple la condición, se rechaza la operación y no se registra una venta parcial.

## Roles y método de colaboración

| Rol de trabajo | Responsabilidades incluidas en el plan |
|---|---|
| Integrante 1 - Lead Developer / Backend y seguridad | Lógica de ventas y stock, esquema relacional, autenticación, controles OWASP y revisión técnica. |
| Integrante 2 - Scrum Master / Frontend, QA y cumplimiento | Coordinación del backlog, interfaz, ejecución de pruebas y documentación ética, legal e ISO. |
| Trabajo conjunto | Validación de criterios, revisión de cambios y consolidación de la entrega. |

La extensión organiza el trabajo en cuatro ciclos: preparación y arquitectura; prototipado y seguridad; testing y refactorización; auditoría y cierre. Este informe describe ese cronograma como plan de trabajo y atribuye como implementado solo lo demostrado por el prototipo y las pruebas adjuntas.

# 5. Prototipo, seguridad OWASP y alcance legal

El prototipo funcional se implementa con Flask y SQLite. Permite registrar prendas de ropa interior por nombre o modelo, talla, precio y stock; consultar el stock disponible al iniciar una venta; calcular el total; y revisar las diez ventas más recientes. El navegador recibe mensajes claros y las plantillas escapan el contenido dinámico.

## Controles de seguridad incorporados

| Riesgo / referencia OWASP Top 10:2025 | Control presente | Prueba o evidencia |
|---|---|---|
| A01 Control de acceso roto; A07 Fallas de autenticación | Inicio de sesión con hash de contraseña; rutas protegidas; cookie HttpOnly y SameSite. | Acceso sin sesión redirige a login; prueba de acceso. |
| A02 Configuración insegura; A04 Fallas criptográficas | debug=False; secreto de sesión y hash fuera del código; cabeceras CSP, nosniff y frame-ancestors. | Revisión de configuración y prueba de cabeceras. |
| A05 Inyección | Consultas parametrizadas y escape HTML automático de Jinja. | Entrada con sintaxis similar a SQL permanece como texto y no elimina la tabla. |
| CSRF / diseño inseguro | Token CSRF obligatorio en todas las operaciones POST. | POST sin token recibe HTTP 400. |
| Condición de carrera / integridad de datos | Descuento condicionado a stock suficiente dentro de transacción inmediata. | Prueba de venta válida y prueba de venta mayor al stock disponible. |

El OWASP Top 10:2025 sirve para priorizar riesgos, mientras que OWASP ASVS ofrece requisitos verificables para controles técnicos. El prototipo aplica controles seleccionados; no afirma cubrir todos los riesgos del Top 10 ni obtener conformidad ASVS.

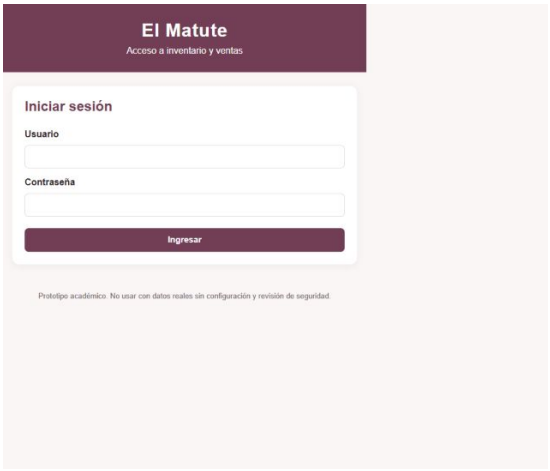
## Corrección Antes / Después

| Antes - aplicación inicial | Después - versión EVA3 |
|---|---|
| Clave fija para sesión en código y modo de depuración activo. | Secreto de sesión desde variable de entorno con valor aleatorio de respaldo para ejecución local; `debug=False`. En producción se debe configurar secreto persistente y rotarlo. |
| Formulario POST sin autenticación ni token CSRF. | Formulario exige sesión y token sincronizado; el servidor compara el token en tiempo constante y rechaza su ausencia. |
| El registro de venta descontaba stock de forma condicional; no había evidencia automatizada para flujo de venta. | Se conserva el descuento condicional y se prueba con escenarios de venta válida y stock insuficiente; se ejecuta como transacción. |

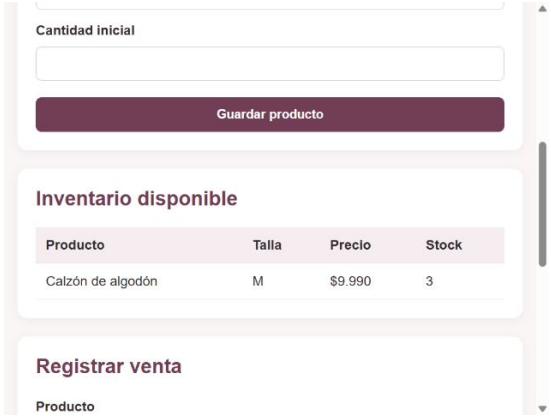
Se conservó el uso de consultas parametrizadas que ya existía. La mejora se concentra en sesiones, CSRF, configuración segura y evidencia repetible de integridad de inventario.

# Evidencia visual del prototipo

Las capturas se tomaron desde el navegador en la versión local del prototipo el 20-10-2026. El producto y la venta son datos ficticios de demostración; no corresponden a registros reales de El Matute.




Inicio de sesión.




Inventario tras registrar venta: stock 3.

## Registro de venta




Venta demo número 1: dos calzones de algodón a $9.990 cada uno, total $19.980; stock actualizado de 5 a 3.

# 6. Testing, ISO 27000 y refactorización

El plan combina pruebas de integración y seguridad sobre una base temporal. Los resultados que siguen se obtuvieron ejecutando `python -m unittest discover -s tests -v` desde la carpeta eva3_entrega el 20-10-2026. La suite no prueba carga, disponibilidad, recuperación de respaldos, gestión de secretos en cloud, escaneo de dependencias ni penetración externa.

## Matriz y resultado de pruebas

| ID / tipo | Procedimiento y resultado esperado | Resultado ejecutado |
|---|---|---|
| T01 - acceso | Abrir inventario sin sesión; se espera redirección a login. | APROBADA |
| T02 - integración | Crear un calzón de algodón talla M a $9.990, stock 5; vender 2; stock 3 y total = 19.980 CLP. | APROBADA |
| T03 - integridad | Intentar vender 6 unidades con stock 5; no cambia stock ni crea venta. | APROBADA |
| T04 - CSRF | Enviar alta de producto sin token; se espera HTTP 400. | APROBADA |
| T05 - inyección | Usar texto con sintaxis similar a SQL en el nombre; se guarda como texto y tabla sigue disponible. | APROBADA |
| T06 - configuración | Comprobar CSP, X-Frame-Options y X-Content-Type-Options en respuesta. | APROBADA |

Resultado: 6 de 6 pruebas automatizadas aprobadas. Los casos están en `tests/test_app.py`; la salida de esta ejecución se adjunta como `evidencias/pruebas_ropa_interior.txt`.

## Análisis según ISO/IEC 27001 y 27002

| Hallazgo / activo | Riesgo inicial | Control alineado | Acción / estado |
|---|---|---|---|
| Acceso a ventas e inventario | Alto: terceros podrían alterar existencias. | Control de acceso, autenticación y gestión de privilegios; ISO/IEC 27002 5.15, 5.17, 5.18 y 8.5. | Sesión requerida y hash de clave implementados; pendiente separar roles y revisar privilegios. |
| Sesión web y formularios | Medio-alto: CSRF y configuración débil. | Desarrollo seguro y codificación segura; 8.25, 8.28 y 8.29. | Token CSRF, cookies seguras y cabeceras añadidos; activar cookie Secure en HTTPS. |
| Base de inventario | Alto: error de venta o pérdida de stock. | Respaldo 8.13, registro 8.15 y gestión de vulnerabilidades 8.8. | Transacción y log del servidor; diseñar backups cloud y prueba de restauración. |
| Dependencias y servicio cloud | Medio: vulnerabilidad o indisponibilidad de proveedor. | Gestión de servicios cloud 5.23 y gestión de vulnerabilidades 8.8. | Inventariar versiones, aplicar parches, revisar SLA y probar salida/migración. |

ISO/IEC 27001 define requisitos para un sistema de gestión de seguridad de la información; ISO/IEC 27002 ofrece controles de referencia. Esta matriz usa controles como guía de mejora y no declara certificación ni cumplimiento formal.

# Referencias

## Normas y estándares consultados

[1] Biblioteca del Congreso Nacional de Chile. Ley 19.628, texto vigente al 30 de noviembre de 2026. Texto oficial.

[2] Biblioteca del Congreso Nacional de Chile. Ley 21.719, modifica la Ley 19.628; vigencia de las modificaciones desde 1 de diciembre de 2026. Texto oficial.

[3] Biblioteca del Congreso Nacional de Chile. Ley 21.459 sobre delitos informáticos, versión vigente consultada al 1 de abril de 2025. Texto oficial.

[4] ACM. Code of Ethics and Professional Conduct, secciones sobre bien público, privacidad, acceso autorizado y seguridad. Código.

[5] OWASP Foundation. OWASP Top 10:2025. Proyecto oficial; OWASP ASVS 5.0.0. Proyecto oficial.

[6] ISO. ISO/IEC 27001:2022, Information security management systems. Página oficial; ISO/IEC 27002:2022, controles de seguridad. Página oficial.

[7] Google Cloud. Cloud SQL, servicio de base de datos relacional administrado. Documentación del proveedor; Cloud Run. Documentación del proveedor.

## Evolución recomendada para El Matute

| Prioridad futura | Motivo y alcance sugerido |
|---|---|
| Validar catálogo y reglas de venta | Confirmar modelos, tallas, cambios y devoluciones con la dueña antes de ampliar el sistema. |
| Despliegue y continuidad | Configurar HTTPS, base administrada, respaldos y pruebas de restauración antes de usar datos reales. |
| Clientes y medios de pago (fuera del prototipo) | Incorporarlos solo si el negocio lo necesita; definir minimización de datos y usar una pasarela externa sin almacenar datos de tarjeta. |
| Auditoría de movimientos (fuera del prototipo) | Si se requiere historial resistente a modificaciones, diseñar permisos, bitácora y respaldos separados; el historial actual no es inmutable. |

Las capturas y resultados de pruebas incluidos corresponden al prototipo académico. Los módulos señalados como evolución no están implementados ni se presentan como capacidades actuales.

Fuentes revisadas el 20 de octubre de 2026.